DP-LMI package
User Manual

Version v0.2.0

July 2026

Abstract

This manual describes the current public behavior of the DP-LMI package. The package uses `dpbase` as the backend parent for cell-local Bernstein coefficient storage, provides `dpmat` for known finite real matrix functions on tensor grids, provides `dpvar` for YALMIP-backed Bernstein decision expressions and `rhodiff` derivatives, and provides `dplmi` for direct coefficient-wise YALMIP constraint assembly. The manual is written as a reference manual: conceptual Bernstein material comes first, and each class chapter then documents supported call forms, arguments, outputs, examples, and limits.

Contents

1

Bernstein And Grid Background

1.1 From Approximation Theory To DP-LMIs

Bernstein introduced this polynomial basis in 1912 in a constructive proof of the Weierstrass approximation theorem. The same control-point representation later became central to Bézier curves and the de Casteljau algorithm [1]. For this package, however, the important bridge to control is not approximation alone. It is the combination of four facts: nonnegative basis functions, a partition of unity, tensor-product coordinates on boxes, and exact coefficient algebra. Together they turn a matrix polynomial on each physical cell into a convex combination of finitely many matrix coefficients.

This viewpoint separates three layers that are easy to conflate. A physical grid divides the scheduling domain into cells. A Bernstein basis represents a polynomial exactly inside each cell. Finally, `dplmi` converts a matrix-sign requirement into finitely many sufficient coefficient constraints. The first two layers are representation; the third is a certificate.

1.2 Local Coordinates And Degree-Two Coefficients

On one physical interval $[\rho_i, \rho_{i+1}]$, the package uses the local coordinate

$$a = \frac{\rho_{i+1} - \rho}{\rho_{i+1} - \rho_i}.$$

Thus $a = 1$ at the lower endpoint and $a = 0$ at the upper endpoint. A degree-m Bernstein polynomial on that cell is

$$P(\rho) = \sum_{j=0}^{m} \binom{m}{j} a^{m-j}(1-a)^j C_j.$$

Write

$$B_j^m(a) = \binom{m}{j} a^{m-j}(1-a)^j.$$

On $0 \leq a \leq 1$, every $B_j^m(a)$ is nonnegative and

$$\sum_{j=0}^{m} B_j^m(a) = (a + (1-a))^m = 1.$$

Thus the Bernstein basis is a nonnegative partition of unity. It also interpolates the cell endpoints in the package ordering: $P(\rho_i) = C_0$ and $P(\rho_{i+1}) = C_m$. Many references instead use $t = (\rho - \rho_i)/(\rho_{i+1} - \rho_i) = 1 - a$. Their forward-index formulas describe the same basis, but endpoint labels must be reversed before comparing coefficients with `LocalValues`.

The degree-two case is the first case where the middle Bernstein scale matters:

$$P(\rho) = a^2 C_0 + 2a(1-a)C_1 + (1-a)^2 C_2.$$

The package stores the coefficient matrices C_0, C_1, C_2, not sampled values of the expanded monomial form. For matrix-valued objects, each C_j is a finite real matrix for `dpmat`, or a YALMIP affine matrix expression for `dpvar`.

For tensor grids, a local Bernstein coefficient label is a row vector. For example, a two-parameter degree-one cell has labels $[0,0]$, $[0,1]$, $[1,0]$, and $[1,1]$. Each label selects one tensor-product Bernstein basis term, one local index per parameter direction.

For ℓ parameters, the tensor-product basis is

$$B_{\boldsymbol{j}}^{m}(\boldsymbol{a}) = \prod_{r=1}^{\ell} B_{j_r}^{m}(a_r).$$

Its terms are again nonnegative and sum to one because

$$\sum_{\boldsymbol{j}} B_{\boldsymbol{j}}^{m}(\boldsymbol{a}) = \prod_{r=1}^{\ell} \sum_{j_r=0}^{m} B_{j_r}^{m}(a_r) = 1.$$

The package uses this box, or hyperrectangular, tensor product. Bernstein bases on simplices are mathematically related, but they are not the storage convention implemented by `dpbase`.

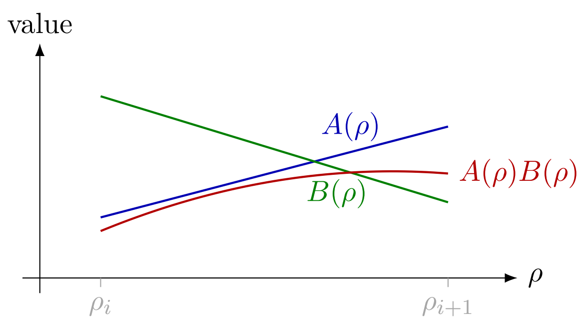

Figure 1: Local Bernstein multiplication raises polynomial degree.

1.3 Convex Hulls And Matrix Sign Certificates

The partition-of-unity identity makes every value a convex combination of its coefficient data:

$$P(\boldsymbol{a}) = \sum_{\boldsymbol{j}} B_{\boldsymbol{j}}^{m}(\boldsymbol{a}) C_{\boldsymbol{j}} \in \operatorname{conv}\{C_{\boldsymbol{j}}\}.$$

For a scalar polynomial this gives the range enclosure

$$\min_{\boldsymbol{j}} C_{\boldsymbol{j}} \le P(\boldsymbol{a}) \le \max_{\boldsymbol{j}} C_{\boldsymbol{j}}.$$

For symmetric matrices, apply the same identity to an arbitrary vector x:

$$x^{\mathsf{T}} P(\boldsymbol{a}) x = \sum_{\boldsymbol{j}} B_{\boldsymbol{j}}^{m}(\boldsymbol{a}) x^{\mathsf{T}} C_{\boldsymbol{j}} x.$$

Consequently, $C_{\boldsymbol{j}} \preceq 0$ for every local label is sufficient for $P(\boldsymbol{a}) \preceq 0$ throughout the cell; the positive-semidefinite statement is analogous. If all finitely many coefficients are strictly negative definite, their smallest eigenvalue margins also give a uniform strict certificate on the cell.

The implication is one-way. A matrix polynomial may be negative definite at every point even though one Bernstein coefficient is not. A failed direct coefficient test therefore does not establish infeasibility of the continuous DP-LMI. This distinction is the central source of conservatism in direct Bernstein assembly and is already present in Bernstein range tests used for interval-parameter robustness [2].

1.4 Multiplication, Degree Growth, And Convolution

Multiplication raises Bernstein degree. In one parameter, multiplying two degree-one factors gives a degree-two product:

$$(aA_0 + (1-a)A_1)(aB_0 + (1-a)B_1) = a^2 A_0 B_0 + a(1-a)(A_0 B_1 + A_1 B_0) + (1-a)^2 A_1 B_1.$$

Because the degree-two Bernstein middle basis is $2a(1-a)$, the stored middle coefficient is $(A_0 B_1 + A_1 B_0)/2$.

This is not a special rule for linear factors. If $B_i^m(a)$ denotes the i-th Bernstein basis function of degree m, then

$$B_i^m(a) B_j^n(a) = \frac{\binom{m}{i}\binom{n}{j}}{\binom{m+n}{i+j}} B_{i+j}^{m+n}(a).$$

For coefficient arrays this is a convolution over coefficient labels. In one parameter,

$$C_k = \sum_{\substack{i+j=k \\ 0 \le i \le m,\ 0 \le j \le n}} A_i B_j \frac{\binom{m}{i}\binom{n}{j}}{\binom{m+n}{k}}, \qquad k = 0, \ldots, m+n.$$

In ℓ parameters, replace the scalar labels i, j, k by multi-indices α, β, γ, use $\alpha + \beta = \gamma$, and multiply the binomial scale over parameter directions:

$$C_\gamma = \sum_{\alpha+\beta=\gamma} A_\alpha B_\beta \prod_{r=1}^{\ell} \frac{\binom{m}{\alpha_r}\binom{n}{\beta_r}}{\binom{m+n}{\gamma_r}}.$$

This is the coefficient rule used by `dpmat * dpmat` and by supported known-data products involving `dpvar`. Matrix order is preserved inside every convolution term: in general $A_\alpha B_\beta \neq B_\beta A_\alpha$, so exchanging the operands changes both the algebra and the resulting coefficient matrices.

1.5 Degree Elevation And Polynomial Representation

A polynomial represented in Bernstein degree m can also be represented in any higher degree $M \ge m$. In one parameter, the elevated coefficient $\widehat{C}_k$ of degree M is

$$\widehat{C}_k = \sum_{i=\max(0,k-(M-m))}^{\min(m,k)} C_i \frac{\binom{m}{i}\binom{M-m}{k-i}}{\binom{M}{k}}, \qquad k = 0, \ldots, M.$$

Tensor grids apply the same formula by tensor product over each parameter direction. The package uses degree elevation when two coefficient-backed objects with the same grid bounds but different Bernstein degree must be combined by addition, subtraction, concatenation, assignment, or other coefficient-wise operations.

Degree elevation does not change the polynomial or add decision freedom. Each elevated coefficient is a convex combination of old coefficients, and repeated elevation makes the coefficient polygon track the represented function more closely [1]. This is different from constructing a new decision function with a higher degree. The current public `dpvar` constructor accepts only `Degree=0` and `Degree=1`; products may nevertheless produce higher-degree expressions, which are aligned internally by `bernElev`.

1.6 Differentiation And de Casteljau Subdivision

The reversed local coordinate changes the intermediate sign in a derivative. For cell width $h = \rho_{i+1} - \rho_i$, the two sign reversals combine to give the forward physical difference

$$\frac{dP}{d\rho} = \frac{m}{h} \sum_{j=0}^{m-1} (C_{j+1} - C_j) B_j^{m-1}(a).$$

This is the scalar-cell formula behind `rhodiff`. In several scheduling dimensions, the directional rate term is $\dot{P} = \sum_{r=1}^{\ell} (\partial P / \partial \rho_r) \dot{\rho}_r$. The implementation forms each cell-local partial difference, elevates partials to a common tensor degree when necessary, and evaluates the affine rate dependence at all active $\dot{\rho}$-box vertices. Derivative boundary rows remain cell-local rather than sharing the continuous coefficient handles of the original `dpvar`.

The de Casteljau recursion evaluates the same polynomial through neighboring affine combinations:

$$C_j^{(r)} = a C_j^{(r-1)} + (1-a) C_{j+1}^{(r-1)}, \qquad C_j^{(0)} = C_j.$$

Its edge chains also give exact Bernstein coefficients on the two subintervals created at the split point. Applying this one-dimensional operation along each parameter axis gives exact tensor-product subdivision. Subdivision is therefore a useful certificate-refinement idea: the polynomial is unchanged, but its convex hulls are recomputed on smaller physical cells. The current package does not expose a general public de Casteljau or adaptive-subdivision API; these facts explain the representation and future refinement direction, not an implemented call form.

1.7 Storage And Hypercube Traversal

The physical grid and the local Bernstein labels are separate. For ℓ parameters with node counts $N_1, \ldots, N_\ell$, the number of physical hypercubes is

$$\prod_{r=1}^{\ell} (N_r - 1).$$

For Bernstein degree m, each physical hypercube stores

$$(m+1)^\ell$$

local coefficients. The helper ordering used by `cells` and `lbls` is lexicographic with earlier dimensions varying more slowly.

Physical cells are stored as nested `LocalValues` entries, indexed by one physical-cell subscript per scheduling dimension. The selected entry is a flat coefficient cell in local label order. For a continuous `dpvar`, adjacent cells reuse the symbolic handles on their common face; continuity is therefore encoded by coefficient sharing, not by adding equality LMIs. Known `dpmat` data follow the same cell-local layout. Discontinuous derivative objects intentionally keep separate boundary entries so each cell retains its own physical-width and rate-vertex derivative data.

Example

The storage counts follow directly from the physical grid and the local Bernstein degree.

MATLAB input

```
A = dpmat({[0 1 2], [10 20]}, {1 2; 3 4; 5 6}, Degree=1);
A.ncell()
A.ncoeff()
A.npar()
```

Output

```
ans =
     2

ans =
     4

ans =
     2
```

The same object exposes the local Bernstein labels and physical-cell traversal order used throughout the package.

MATLAB input

```
A = dpmat({[0 1 2], [10 20]}, {1 2; 3 4; 5 6}, Degree=1);
A.lbls()
A.cells()
```

Output

```
ans =
     0     0
     0     1
     1     0
     1     1

ans =
     1     1
     2     1
```

For a short coefficient display, `bernsteinTable(A, "oneLine")` prints the local Bernstein expression instead of every metadata column:

MATLAB input

```
A = dpmat({[0 1]}, {1, 3}, Degree=1);
T = bernsteinTable(A, "oneLine");
disp(T)
```

Output

```
CellSubscript      Expression
-------------    ---------------

    {[1]}        "a*1 + (1-a)*3"
```

1.8 Conservatism And Refinement Hierarchy

When a coefficient-wise LMI fails, the following distinctions keep the next modeling step precise:

1. The failure proves only that the present sufficient certificate did not succeed; it does not prove the continuous DP-LMI infeasible.
2. Physical cell subdivision restricts and reparameterizes the same polynomial on smaller cells, usually tightening local convex hulls.
3. Pure degree elevation preserves exactly the same polynomial and decision variables. A genuinely higher-degree decision parameterization instead enlarges the search space.
4. Vertex-pair, Polya, or sum-of-squares constructions are different sufficient hierarchies. They require their own theorems, auxiliary variables, options, and tests.

Spline-type parameter-dependent LMI methods give important motivation for partition refinement. Masubuchi, Kume, and Shimemura derived finite local LMI conditions and an asymptotic refinement result under the assumptions of their formulation [3]. Xu and Jabbari continued the grid/spline line in parameter-varying output-feedback synthesis and improved L_2-gain computation [4]. These results motivate the package's cell-local architecture, but they are not a blanket completeness theorem for every package mode.

1.9 Control Context And The Implemented Boundary

The literature places this representation among several complementary polynomial-control certificates. Zettler and Garloff used multivariate Bernstein expansion and convex-hull bounds for robustness analysis of polynomial families with interval parameters [2]. Chesi's survey situates coefficient tests beside broader LMI techniques for polynomial optimization in control, including homogeneous and sum-of-squares approaches [5]. The present package is related to that lineage, but its cell-local tensor storage and rate-vertex assembly are project-specific.

The implemented public boundary is deliberately narrower than this background:

- `dpbase` provides backend Bernstein storage, degree elevation, and product convolution used by `dpmat` and `dpvar`.
- `rhodiff` forms the implemented rate-affine derivative rows, and `dplmi` directly constrains every active rate row and local coefficient before `toYalmip` hands the model to YALMIP.
- Relaxation-lemma assembly, Polya assembly, sum-of-squares machinery, adaptive subdivision, package-owned strictness margins, residual certificates, and diagnostic hierarchies are not current public APIs. Reserved nondefault `dplmi` options are rejected rather than silently approximated.

2

Installation, Verification, And Lookup

2.1 Installation

Start MATLAB in the repository root, or set `projectRoot` to the absolute path of the checked-out repository. Add the package recursively, then remove the documentation directory from the executable path.

MATLAB input

```
projectRoot = "path/to/Differential Parameter—Dependent Linear Matrix Inequality";
addpath(genpath(projectRoot));
rmpath(genpath(fullfile(projectRoot, "doc")));
```

For local work from the repository root, this package also resolves from:

MATLAB input

```
addpath(pwd)
```

2.2 Verification

Run the current MATLAB test entry point from the repository root:

MATLAB input

```
results = tests.run_all();
```

The test entry point covers helper utilities, `dpbase`, `dpmat`, `dpvar`, and `dplmi`. The YALMIP-backed `dpvar` and `dplmi` tests require YALMIP. Solver smoke tests additionally require an SDP solver.

2.3 Quick Start

MATLAB input

```
A = dpmat({[0 1]}, {1, 3}, Degree=1);
T = bernsteinTable(A, "oneLine");
disp(T)
```

Output

```
    CellSubscript      Expression
    _____________    _______________

        {[1]}        "a*1 + (1-a)*3"
```

MATLAB input

```
yalmip('clear')
P = dpvar(2, {[0 1]}, "symmetric");
C = P >= 0;
F = toYalmip(C);
```

`F` can be passed to ordinary YALMIP calls, for example:

Call forms

```
sol = optimize(F, objective, opts);
```

2.4 Global Function Lookup

Category	Entries
Backend	`dpbase`, constructor, `cells`, `lbls`, `coeffs`, `ncell`, `ncoeff`, `npar`, `size`
Known data	`dpmat`, construction, `evaluate`, `bernsteinTable`, `disp/display`, `plot`, operators, matrix methods
Decision variables	`dpvar`, construction, `rhodiff`, affine/mixed algebra, matrix methods, comparisons
LMIs	`dplmi`, construction, direct coefficient assembly, `toYalmip`, solver examples

3

dpbase Backend Reference

`dpbase` is the developer-facing backend parent for `dpmat` and `dpvar`. Ordinary users should model known data with `dpmat` and decision expressions with `dpvar`. This section documents `dpbase` because it explains the shared storage, traversal, and inspection contract used by both public modeling classes.

3.1 Lookup

Task	Function or field
Create backend object	`dpbase`
Inspect cells/labels	`cells`, `lbls`, `coeffs`
Count sizes	`ncell`, `ncoeff`, `npar`, `size`
Read storage fields	`GridInfo`, `LocalValues`, metadata fields

3.2 `dpbase` Constructor

Syntax

Call forms

```
obj = dpbase(gridVectors, matrixSize, degree)
obj = dpbase(gridVectors, matrixSize, degree, localValues)
obj = dpbase(..., IsContinuous=tf)
obj = dpbase(..., ContainsDecision=tf)
obj = dpbase(..., HasRateDependence=tf)
obj = dpbase(..., RateBounds=rb)
obj = dpbase(..., SourceSummary=txt)
```

Arguments and options

- `gridVectors` is a cell vector with one strictly increasing node vector per parameter, such as `\{[0 1 2], [10 20]\}`.
- `matrixSize` is a positive two-element matrix size, such as `[2 2]`.
- `degree` is a nonnegative integer Bernstein degree, such as `1`.
- `localValues` is the nested physical-cell storage. When omitted, `dpbase` creates a zero coefficient-backed object.

- `RateBounds` is empty or an $\ell \times 2$ matrix of finite lower and upper rate bounds, such as `RateBounds=[-1 1; -2 2]`.

Example

This backend example constructs a two-parameter parent object. The object has two physical cells because the first parameter has two intervals and the second parameter has one interval.

MATLAB input

```
obj = dpbase({[0 1 2], [10 20]}, [2 2], 1);
class(obj)
obj.MatrixSize
obj.Degree
obj.ncell()
obj.ncoeff()
obj.npar()
size(obj)
```

Output

```
ans =
    'dpbase'

ans =
     2     2

ans =
     1

ans =
     2

ans =
     4

ans =
     2

ans =
     2     2
```

The printed `ncoeff` value is $(1+1)^2 = 4$, one local Bernstein coefficient for each tensor-product label in the two parameter directions.

3.3 Storage Fields

Field	Meaning
`GridInfo`	Grid vectors, node counts, and grid bounds.
`MatrixSize`	Size of each matrix coefficient or matrix expression.
`Degree`	Local Bernstein degree stored in every physical cell.
`LocalValues`	Nested cell tree indexed by physical-cell subscript; each leaf stores coefficient cells.
`IsContinuous`	Metadata flag; subclasses are responsible for boundary sharing if continuity is true.
`ContainsDecision`	True when coefficient payloads include YALMIP decision variables.
`HasRateDependence`	True when the object carries rate metadata or rate-vertex rows.
`RateBounds`	Parameter-rate lower and upper bounds used by `rhodiff` and `dplmi`.
`SourceSummary`	Short origin label such as `coefficient-backed`, `decision`, or `derivative`.

3.4 Inspection: `cells`, `lbls`, And `coeffs`

Syntax

Call forms

```
subs = cells(obj)
subs = obj.cells()

labels = lbls(obj)
labels = obj.lbls()

c = coeffs(obj, cellSubs)
c = obj.coeffs(cellSubs)
```

Arguments and options

`cells` returns physical hypercube subscripts in the shared traversal order. `lbls` returns local Bernstein labels in the shared coefficient order. `coeffs(obj, cellSubs)` returns the flat coefficient cell array for one physical cell; `cellSubs` may be a numeric row vector such as `[2 1]` or a cell array of scalar subscripts such as `\{2,1\}`.

Example

The label order is shared by `dpmat`, `dpvar`, and `dplmi`. In this two-parameter degree-one object, each physical cell stores four coefficient entries with labels $[0, 0]$, $[0, 1]$, $[1, 0]$, and $[1, 1]$.

MATLAB input

```
obj = dpbase({[0 1 2], [10 20]}, [1 1], 1);
lbls(obj)
cells(obj)
c = coeffs(obj, [2 1]);
numel(c)
```

Output

```
ans =
     0     0
     0     1
     1     0
     1     1

ans =
     1     1
     2     1

ans =
     4
```

3.5 Counts And Shape

Syntax

Call forms

```
n = ncell(obj)
n = obj.ncell()

n = ncoeff(obj)
n = obj.ncoeff()

n = npar(obj)
n = obj.npar()

sz = size(obj)
d = size(obj, dim)
```

Arguments and options

`ncell` returns $\prod_r (N_r - 1)$. `ncoeff` returns $(m+1)^\ell$. `npar` returns the parameter dimension ℓ. `size` returns the matrix payload size, not the physical grid size.

3.5.1 `bernElev`

Syntax

Call forms

```
out = bernElev(obj, coeffs, fromDeg, toDeg)
```

Arguments and options

`bernElev` degree-elevates one flat cell-local Bernstein coefficient family. The source and target degrees must be nonnegative integers with `toDeg >= fromDeg`; the coefficient-cell length must match the source degree and parameter dimension. The output contains $(\texttt{toDeg} + 1)^{\ell}$ coefficients in the same label order as `lbls`.

Example

MATLAB input

```
A = dpmat({[0 1]}, {[1 2]}, Degree=1);
B = dpmat({[0 1]}, {1, 2, 3}, Degree=2);
C = A + B;
C.Degree
```

Output

```
ans =
     2
```

The example invokes the protected helper through public addition; users do not call `bernElev` directly.

3.5.2 `bernProd`

Syntax

Call forms

```
out = bernProd(obj, lhs, lhsDeg, rhs, rhsDeg)
```

Arguments and options

`bernProd` computes the binomial-normalized, cell-local Bernstein coefficient convolution of two coefficient families. The output degree is `lhsDeg + rhsDeg`; matrix payload multiplication follows MATLAB matrix product rules and is performed coefficient by coefficient.

Example

MATLAB input

```
A = dpmat({[0 1]}, {[1 2]}, Degree=1);
P = A * A;
P.Degree
```

Output

```
ans =
     2
```

The example invokes the protected helper through public matrix multiplication; users do not call `bernProd` directly.

3.5.3 `mergeGrid`

Syntax

Call forms

```
grid = obj.mergeGrid(errId, other1, other2, ...)
```

Arguments and options

`mergeGrid` builds a same-bound common refinement from gridded operands. It preserves the lower and upper bound in every parameter dimension and inserts all interior nodes. `errId` is the identifier used when parameter dimensions or physical bounds are incompatible. This method is primarily a backend utility invoked by public algebra methods.

Example

MATLAB input

```
A = dpmat({[0 0.5 1]}, {1, 2}, Degree=1);
B = dpmat({[0 1]}, {3, 4}, Degree=1);
C = A + B;
C.GridInfo.Vectors{1}
```

Output

```
ans =
         0    0.5000    1.0000
```

The example invokes the protected helper through public addition; users do not call `mergeGrid` directly.

3.6 See Also

`dpmat`, `dpvar`, `cells`, `coeffs`, `lbls`, `ncell`, `ncoeff`, `npar`, `size`.

4

dpmat Reference

`dpmat` stores known finite real matrix data on a tensor grid. Objects can be coefficient-backed, function-only, or function-backed with explicit Bernstein coefficient evidence. Coefficient-backed objects participate in algebra. Function-only objects evaluate exactly through the stored handle, but do not expose coefficient evidence for `bernsteinTable` or coefficient algebra.

4.1 Lookup

Task	Entry
Construct data	`dpmat`
Evaluate	`evaluate`
Inspect/display	`bernsteinTable`, `disp`, `display`
Plot	`plot`
Arithmetic	`+`, `-`, unary `+/-`, `*`
Matrix/index methods	shape and structural methods, indexing and assignment

4.2 Construction

Syntax

Call forms

```
A = dpmat(gridVector, source)
A = dpmat(gridVector, source, Degree=m)
A = dpmat(gridVectors, source)
A = dpmat(gridVectors, source, Degree=m)
```

Arguments and options

- `gridVector` is a numeric vector used as a one-parameter grid shorthand, such as `[0 1 2]`.
- `gridVectors` is a cell vector with one numeric grid vector per parameter, such as `\{[0 1], [10 20]\}`.
- `source` may be a function handle, a global cell grid of numeric Bernstein coefficients, or explicit nested `LocalValues`; for example, `\{1, 2, 3\}` is scalar degree-two data on one cell.
- `Degree=m` sets the local Bernstein degree. For function handles, omitting `Degree` creates a function-only object; passing a value such as `Degree=2` validates and stores Bernstein coefficient

evidence while retaining the exact function handle for `evaluate`.

Example

The scalar-grid shorthand is the shortest coefficient-backed form. The three numeric cells below are the global Bernstein coefficients for two physical cells with degree one.

MATLAB input

```
A = dpmat([0 1 2], {10, 20, 30}, Degree=1)
size(A)
A.Degree
A.SourceSummary
```

Output

```
A =
  dpmat [1 1] over 1-D grid, degree 1, source coefficient-backed

ans =
     1     1

ans =
     1

ans =
    "coefficient-backed"
```

For tensor grids, pass one grid vector per parameter. The source cell array matches the global coefficient grid implied by the two parameter directions.

MATLAB input

```
B = dpmat({[0 1], [10 20]}, {1 2; 3 4}, Degree=1)
B.GridInfo.Vectors{2}
B.lbls()
```

Output

```
B =
  dpmat [1 1] over 2-D grid, degree 1, source coefficient-backed

ans =
    10    20

ans =
     0     0
     0     1
     1     0
     1     1
```

Use explicit nested `LocalValues` when each physical cell must be written directly. Here each cell stores two 1×2 row-vector coefficients.

MATLAB input

```
localVals = {{[1 0], [0 1]}, {[2 0], [0 2]}};
C = dpmat({[0 1 2]}, localVals, Degree=1)
size(C)
C.coeffs(2)
```

Output

```
C =
  dpmat [1 2] over 1-D grid, degree 1, source coefficient-backed

ans =
     1     2

ans =
  1x2 cell array
    {[2 0]}    {[0 2]}
```

When `source` is a function handle and `Degree` is omitted, the object keeps the exact evaluator but does not claim coefficient evidence.

MATLAB input

```
F = dpmat({[0 1]}, @(rho) [rho rho^2])
F.SourceSummary
F.evaluate(0.5)
```

Output

```
F =
  dpmat [1 2] over 1-D grid, degree 1, source function

ans =
    "function"

ans =
    0.5000    0.2500
```

Add `Degree` for a polynomial function when later coefficient inspection or algebra should use Bernstein evidence. The exact function handle is still used for point evaluation.

MATLAB input

```
G = dpmat({[0 1]}, @(rho) rho.^2, Degree=2)
G.SourceSummary
G.coeffs(1)
```

Output

```
G =
  dpmat [1 1] over 1-D grid, degree 2, source function-bernstein

ans =
    "function-bernstein"

ans =
  1x3 cell array
    {[0]}    {[0]}    {[1]}
```

These forms all create finite known-data objects, but only the coefficient- backed and function-Bernstein forms can participate in coefficient algebra.

4.3 `evaluate`

Syntax

Call forms

```
val = evaluate(A, point)
val = A.evaluate(point)
```

Arguments and options

`point` is a scalar for a one-parameter object, such as `0.25`, or a row vector with one entry per parameter, such as `[0.25 12]`. The return value is a numeric matrix with size `size(A)`. Coefficient-backed objects use local Bernstein interpolation. Function-backed objects evaluate through their stored function handle.

For a degree-one scalar coefficient-backed object on $[\rho_0, \rho_1]$,

$$A(\rho) = aA_0 + (1-a)A_1, \qquad a = \frac{\rho_1 - \rho}{\rho_1 - \rho_0}.$$

The same local-coordinate rule is applied entrywise for matrix-valued data.

Example

Use the function-call form when the object is naturally one input among several MATLAB expressions.

MATLAB input

```
A = dpmat({[0 1]}, {2, 4}, Degree=1);
val = evaluate(A, 0.25)
```

Output

```
val =
    2.5000
```

At $\rho = 0.25$, the local weight is $a = 0.75$, so the interpolated value is $0.75 \cdot 2 + 0.25 \cdot 4 = 2.5$.

The dot-call form is equivalent and can be convenient in command-window exploration.

MATLAB input

```
A = dpmat({[0 1]}, {2, 4}, Degree=1);
val = A.evaluate(0.75)
```

Output

```
val =
    3.5000
```

Function-backed objects route through the retained function handle, so non-polynomial expressions can still be evaluated exactly at a point.

MATLAB input

```
F = dpmat({[0 pi]}, @(rho) sin(rho));
val = F.evaluate(pi/2)
```

Output

```
val =
     1
```

4.4 Visualization And Inspection

4.4.1 `bernsteinTable`

Syntax

Call forms

```
T = bernsteinTable(A)
T = bernsteinTable(A, cellSubs)
T = bernsteinTable(A, "oneLine")
T = bernsteinTable(A, cellSubs, "oneLine")
```

Arguments and options

- `cellSubs` selects one physical cell.
- `"oneLine"` returns one row per selected physical cell with a compact Bernstein expression.
- Without `"oneLine"`, the full table uses seven metadata columns; the output below shows their exact MATLAB names.

For one parameter, the `Basis` column represents

$$\binom{m}{j} a^{m-j}(1-a)^j.$$

For multiple parameters, the basis is the tensor product of that expression in each local coordinate. `LocalIndex` is the local Bernstein label. `CoeffSubscript` maps the local label back to the global coefficient subscript on the physical grid. `IsPhysicalNode` is true when that coefficient lies on a physical grid node.

Example

MATLAB input

```
A = dpmat({[0 1]}, {[0 1], [1 2]}, Degree=1);
T = bernsteinTable(A, 1, "oneLine");
disp(T)
```

Output

```
    CellSubscript           Expression
    _____________    ________________________

       {[1]}         "a*[0 1] + (1-a)*[1 2]"
```

Use the full table when the metadata columns are the point of the example:

MATLAB input

```
A = dpmat({[0 1]}, {1, 2, 3}, Degree=2);
T = bernsteinTable(A);
disp(T)
```

Output

```
    TermIndex    CellSubscript    CoeffSubscript    LocalIndex      Basis      IsPhysicalNode    Value
    _________    _____________    ______________    __________    _________    ______________    _____

        1            {[1]}            {[1]}           {[0]}       "a^2"             true         {[1]}
        2            {[1]}            {[2]}           {[1]}       "2a(1-a)"         false        {[2]}
        3            {[1]}            {[3]}           {[2]}       "(1-a)^2"         true         {[3]}
```

4.4.2 `disp` And `display`

Syntax

Call forms

```
disp(A)
display(A)
A
```

Arguments and options

`disp(A)` prints a compact one-line summary. `display(A)` is the command-window display used when an expression is not terminated by a semicolon; it prints grid, degree, coefficient, and source metadata.

Example

MATLAB input

```
A = dpmat({[0 1]}, {1, 2}, Degree=1);
disp(A)
display(A)
```

Output

```
dpmat [1 1] over 1-D grid, degree 1, source coefficient-backed
A =
  dpmat with 1-by-1 matrix values
  Parameters: 1
    rho_1: [0, 1], 2 nodes
  Degree: 1
  Physical cells: 1
  Coefficients per cell: 2
  Continuous: true
  Source: coefficient-backed
  Function handle: false
```

4.4.3 `plot`

Syntax

Call forms

```
h = plot(A)
h = plot(A, dims)
h = plot(A, SamplesPerCell=n)
h = plot(A, dims, SamplesPerCell=n)
h = plot(A, ..., Name, Value)
```

Arguments and options

- `dims` is one or two parameter indices. For one-parameter objects, the default is `1`; otherwise the default is `[1 2]`, as in `plot(A, [1 2])`.
- `SamplesPerCell` is the package-owned sampling option. The default is `15`; use `SamplesPerCell=4` for a coarse diagnostic plot.
- Remaining name-value pairs pass through to MATLAB graphics. One- dimensional plots pass them to `plot`; two-dimensional plots pass them to `surf`. Examples include `LineWidth`, `FaceAlpha`, and `EdgeColor`.
- The output `h` is the vector of graphics handles, one entry per matrix element.

Example

The one-dimensional call returns one graphics handle per matrix entry. The example inspects the generated sample count and line styling without relying on an interactive figure window.

MATLAB input

```
fig = figure('Visible','off');
A = dpmat({[0 1]}, @(rho) [rho, rho^2]);
h = plot(A, SamplesPerCell=4, LineWidth=2);
numel(h)
numel(h(1).XData)
h(1).LineWidth
h(1).YData([1 end])
close(fig)
```

Output

```
ans =
     2

ans =
     5

ans =
     2

ans =
     0     1
```

With four samples per cell on one physical interval, MATLAB stores five plotted node values including both endpoints.

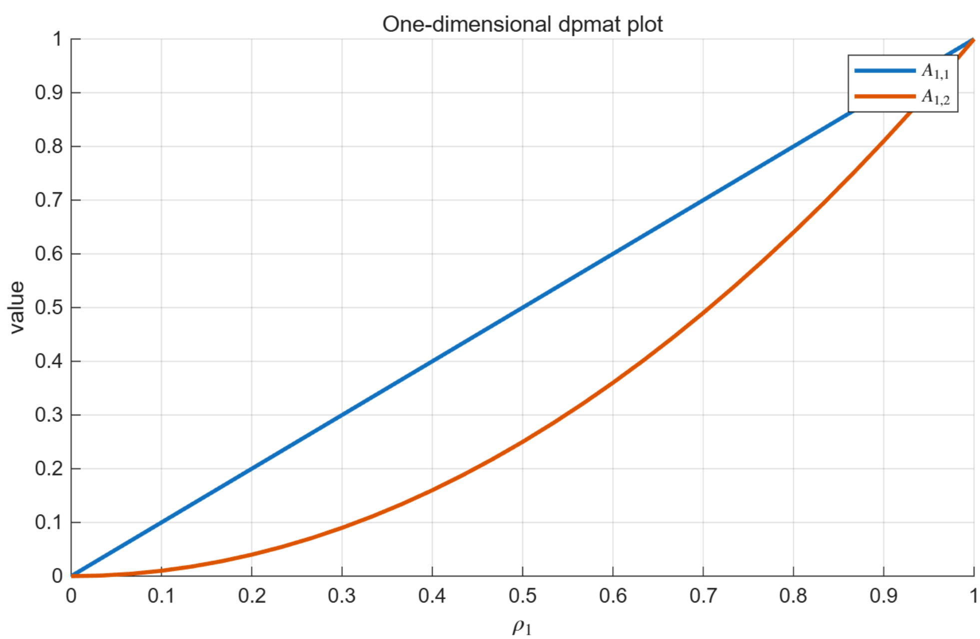

Figure 2: One-dimensional `dpmat/plot` output generated from MATLAB.

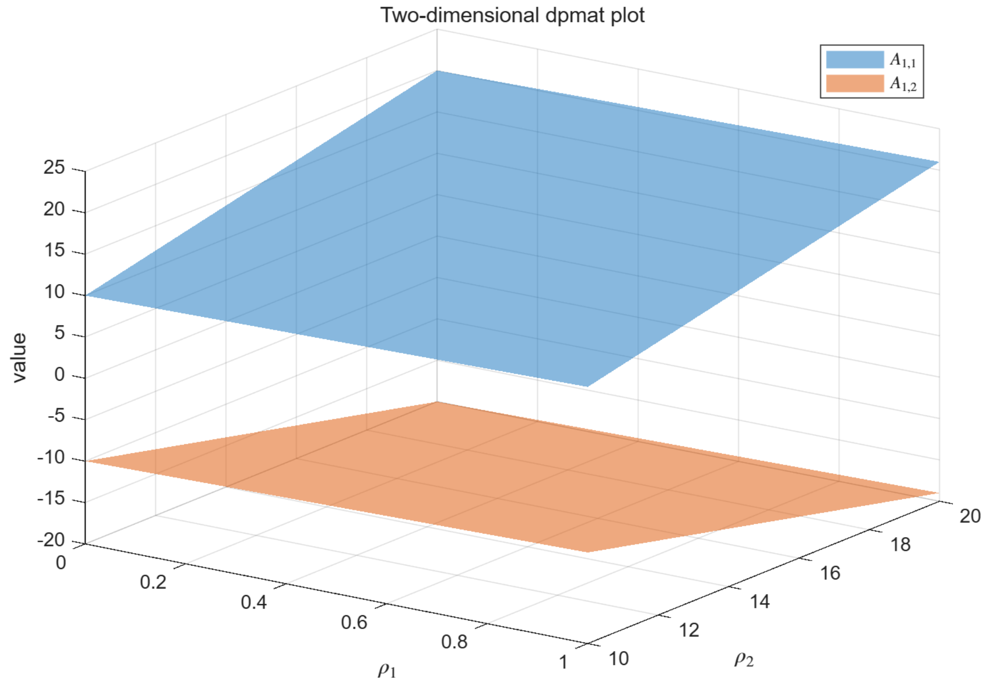

Figure 3: Two-dimensional `dpmat/plot` surface output generated from MATLAB.

4.5 Algebra Operators

Syntax

Call forms

```
C = A + B
C = A + M
C = M + A
C = A - B
C = A - M
C = M - A
C = +A
C = -A
C = A * B
C = A * M
C = M * A
```

Arguments and options

- Coefficient-backed `dpmat`: addition, subtraction, unary signs, and multiplication use local Bernstein coefficients.
- Numeric scalar, such as `5`, broadcasts to the `dpmat` matrix size when needed.

- Numeric matrix: accepted when the matrix size is compatible with the operation, such as `eye(2)` for a 2×2 object.
- Mixed grids with same bounds: re-expressed on a common refinement.
- Mixed degrees: elevated to a common degree for coefficient-wise operations; multiplication produces the degree sum.
- Function-only `dpmat`: evaluates and plots, but does not enter coefficient algebra unless explicit `Degree` supplied coefficient evidence.

For addition and subtraction, the package elevates both operands to a common degree before combining matching coefficients:

$$C_j = \widehat{A}_j + \widehat{B}_j.$$

For multiplication, coefficient labels convolve and the output degree is the sum of the input degrees.

Example

Addition elevates the lower-degree operand before combining coefficients.

MATLAB input

```
A = dpmat({[0 1]}, {1, 2}, Degree=1);
B = dpmat({[0 1]}, {10, 20, 30}, Degree=2);
S = A + B
cS = S.coeffs(1);
[cS{:}]
```

Output

```
S =
  dpmat [1 1] over 1-D grid, degree 2, source coefficient-backed

ans =
   11.0000   21.5000   32.0000
```

Multiplication raises degree from 1 and 2 to 3.

MATLAB input

```
A = dpmat({[0 1]}, {1, 2}, Degree=1);
B = dpmat({[0 1]}, {10, 20, 30}, Degree=2);
P = A * B
cP = P.coeffs(1);
[cP{:}]
```

Output

```
P =
  dpmat [1 1] over 1-D grid, degree 3, source coefficient-backed

ans =
   10.0000   20.0000   36.6667   60.0000
```

Numeric scalars are promoted to compatible constant coefficient data.

MATLAB input

```
A = dpmat({[0 1]}, {1, 2}, Degree=1);
M = 5 - A
N = 2 * A
cM = M.coeffs(1);
cN = N.coeffs(1);
[cM{:}]
[cN{:}]
```

Output

```
M =
  dpmat [1 1] over 1-D grid, degree 1, source coefficient-backed

N =
  dpmat [1 1] over 1-D grid, degree 1, source coefficient-backed

ans =
     4     3

ans =
     2     4
```

These examples use the one-cell case so the coefficient vectors are visible; the same elevation and convolution rules apply cell by cell on tensor grids.

4.5.1 Operator-level entries

`plus`. Adds two coefficient-backed objects or adds a compatible numeric scalar/matrix. The supported forms are `C=A+B`, `C=A+M`, and `C=M+A`; grids are refined and degrees are elevated before coefficient-wise addition. For example, `size(A+3)` returns the same matrix size as `A`.

Syntax

Call forms

```
C = A + B
C = A + M
C = M + A
```

`minus`. Subtracts compatible known-data objects or numeric operands. `A-M` and `M-A` preserve the left/right order of the matrix subtraction.

Syntax

Call forms

```
C = A - B
C = A - M
C = M - A
```

uplus and uminus. Unary plus returns an equivalent coefficient-backed object; unary minus negates every local coefficient.

Syntax

Call forms

```
B = +A
B = -A
```

mtimes. Performs a matrix product, scalar product, or numeric matrix product. The inner matrix dimensions must agree and the output degree is the sum of operand degrees for two parameter-dependent operands.

Syntax

Call forms

```
C = A * B
C = A * M
C = M * A
```

Example

MATLAB input

```
A = dpmat({[0 1]}, {1, 2}, Degree=1);
class(+A)
(—A).evaluate(0)
(A + 3).evaluate(1)
```

Output

```
ans =
    'dpmat'

ans =
    -1

ans =
     5
```

4.6 Matrix Methods

Syntax

Call forms

```
sz = size(A)
n = length(A)
n = height(A)
n = width(A)
n = numel(A)
n = ndims(A)

B = A.'
B = A'
B = vec(A)
B = diag(A)
B = diag(A, k)
B = reshape(A, sz)
B = reshape(A, m, n)
B = tril(A)
B = triu(A)
B = trace(A)
B = sum(A)
B = sum(A, dim)
B = sum(A, "all")
B = mean(A)
B = mean(A, dim)
B = mean(A, "all")
B = cumsum(A)
B = cumsum(A, dim)
B = flip(A)
B = flip(A, dim)
B = flipud(A)
B = fliplr(A)
B = rot90(A)
B = rot90(A, k)
B = repmat(A, ...)
B = [A, B]
B = [A; B]
B = cat(dim, A, B, ...)
B = blkdiag(A, B, ...)
```

```
tf = isequal(A, B, ...)
```

Arguments and options

These methods apply the corresponding MATLAB matrix operation to every local coefficient. They require coefficient evidence. Dimension arguments follow MATLAB conventions, such as `sum(A,2)` for row sums or `reshape(A,[1 4])` for a fixed output shape. Concatenation and `blkdiag` accept compatible coefficient-backed `dpmat` objects and numeric blocks, align grids, and elevate degrees when needed.

Example

Structural methods return new `dpmat` objects whose matrix size changes while the grid and degree metadata stay attached to the coefficient storage.

MATLAB input

```
A = dpmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);
V = vec(A)
size(V)
D = diag(A)
size(D)
R = reshape(A, [1 4])
size(R)
```

Output

```
V =
  dpmat [4 1] over 1-D grid, degree 1, source coefficient-backed

ans =
     4     1

D =
  dpmat [2 1] over 1-D grid, degree 1, source coefficient-backed

ans =
     2     1

R =
  dpmat [1 4] over 1-D grid, degree 1, source coefficient-backed

ans =
     1     4
```

Summary and assembly methods follow the same coefficient-wise rule.

MATLAB input

```
A = dpmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);
Tr = trace(A)
H = [diag(A), diag(A)]
size(Tr)
size(H)
```

Output

```
Tr =
  dpmat [1 1] over 1-D grid, degree 1, source coefficient-backed

H =
  dpmat [2 2] over 1-D grid, degree 1, source coefficient-backed

ans =
     1     1

ans =
     2     2
```

4.6.1 Additional matrix and protocol methods

`transpose` and `ctranspose`. `transpose` applies `A.'`; `ctranspose` applies `A'`. For the finite real data supported by `dpmat`, both return the same numerical transpose.

Syntax

Call forms

```
B = transpose(A)
B = ctranspose(A)
B = A.'
B = A'
```

`squeeze`. Preserves the two-dimensional matrix payload while removing singleton dimensions according to the package's matrix convention.

Syntax

Call forms

```
B = squeeze(A)
```

`horzcat` and `vertcat`. The bracket forms `[A B]` and `[A; B]` concatenate compatible coefficient payloads horizontally or vertically, respectively.

Syntax

Call forms

```
C = horzcat(A, B)
C = vertcat(A, B)
C = [A, B]
C = [A; B]
```

Example

MATLAB input

```
A = dpmat({[0 1]}, {1, 2}, Degree=1);
size([A, A])
size([A; A])
size(squeeze(A))
```

Output

```
ans =
     1     2

ans =
     2     1

ans =
     1     1
```

`subsref` and `subsasgn`. `subsref` implements two-dimensional matrix indexing and dot access to public properties/methods; `subsasgn` implements in-bounds numeric or compatible `dpmat` block assignment.

Syntax

Call forms

```
B = subsref(A, S)
A = subsasgn(A, S, M)
B = A(rows, cols)
A(rows, cols) = M
```

`end`. Supplies the last row or column index for expressions such as `A(1:end,end)`. The related `numArgumentsFromSubscript` method keeps subscripted results scalar so property and method dot access remains available.

Syntax

Call forms

```
idx = end(A, k, n)
n = numArgumentsFromSubscript(A, S, ctx)
B = A(1:end, end)
```

Example

MATLAB input

```
A = dpmat({[0 1]}, {[1 2; 3 4], [5 6; 7 8]}, Degree=1);
size(A(1:end, end))
```

Output

```
ans =
     2     1
```

4.7 Indexing And Assignment

Syntax

Call forms

```
B = A(rows, cols)
value = A.PropertyName
A(rows, cols) = numericValue
A(rows, cols) = B
```

Arguments and options

Matrix indexing slices every local coefficient. Dot access exposes public properties and methods. Assignment accepts numeric blocks and compatible coefficient-backed `dpmat` blocks, with degree elevation when needed. Use ordinary MATLAB matrix subscripts such as `A(:, end)` or `A(1, 2:3)`.

Example

Indexing returns a smaller `dpmat`; assignment updates the corresponding matrix entries in every local coefficient.

MATLAB input

```
A = dpmat({[0 1]}, {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
lastCol = A(:, end)
topTail = A(1, 2:3)
A(:, 2) = 5;
A
```

Output

```
lastCol =
  dpmat [2 1] over 1-D grid, degree 1, source coefficient-backed

topTail =
  dpmat [1 2] over 1-D grid, degree 1, source coefficient-backed

A =
  dpmat [2 3] over 1-D grid, degree 1, source coefficient-backed
```

The assignment does not change the object degree because the numeric value is promoted to degree-one coefficient data before insertion.

4.8 Limits

- Function-only objects created without `Degree` do not have Bernstein coefficient evidence for `bernsteinTable` or coefficient algebra.
- Numeric operands must be finite real nonempty matrices with compatible sizes, or finite real scalars that can broadcast to the required size.
- Grid refinement is supported for matching parameter dimensions and matching grid bounds.

4.8.1 See Also

`dpmat`, `evaluate`, `bernsteinTable`, `plot`, `dpbase`.

5

dpvar Reference

`dpvar` creates continuous cell-local Bernstein decision expressions backed by YALMIP `sdpvar` coefficients. It participates in supported affine and known-data algebra. It does not choose solvers or call `optimize`; solver-facing work happens after comparison overloads create `dplmi` objects and `toYalmip` converts them to YALMIP constraints.

5.1 Lookup

Task	Entry
Construct decisions	`dpvar`
Inspect coefficients	`bernsteinTable`
Differentiate rates	`rhodiff`
Affine/mixed algebra	`+`, `-`, `*` with known data
Matrix/index methods	structural methods, indexing and assignment
Build LMIs	`<=`, `>=`

5.2 Construction

Syntax

Call forms

```
P = dpvar(n, gridVectors)
P = dpvar(n, n, gridVectors)
P = dpvar(n, m, gridVectors)
P = dpvar(..., "full")
P = dpvar(..., "symmetric")
P = dpvar(..., Degree=0)
P = dpvar(..., Degree=1)
P = dpvar(..., RateBounds=rb)
```

Arguments and options

- `dpvar(n, gridVectors)` creates an $n \times n$ square variable using scalar-size shorthand; for example, `dpvar(2, \{[0 1]\})`. Square variables default to symmetric YALMIP coefficients.
- `dpvar(n, n, gridVectors)` is the explicit square-size form; for example, `dpvar(2, 2, \{[0 1]\})`. It has the same default symmetric structure as the shorthand.

- `dpvar(n, m, gridVectors)` creates an $n \times m$ variable. Rectangular variables default to full YALMIP coefficients; for example, `dpvar(2, 3, \{[0 1]\})`.
- `"full"` and `"symmetric"` explicitly choose the YALMIP coefficient structure. `"symmetric"` requires a square size.
- `Degree=1` is the default continuous Bernstein decision variable.
- `Degree=0` creates a parameter-independent variable stored on the grid.
- `RateBounds=rb` stores parameter-rate metadata for later `rhodiff(P)` calls. Use `RateBounds=[-1 1]` for one parameter; use a two-row matrix for two parameters, as shown in the rate-metadata example below.

Example

The square shorthand creates a 2×2 continuous decision object. The default degree is one and the object carries no rate metadata.

MATLAB input

```
yalmip('clear')
Ps = dpvar(2, {[0 1]});
class(Ps)
size(Ps)
Ps.Degree
Ps.HasRateDependence
```

Output

```
ans =
    'dpvar'

ans =
     2     2

ans =
     1

ans =
  logical
   0
```

Use two dimension arguments for a rectangular full variable.

MATLAB input

```
yalmip('clear')
Pf = dpvar(2, 3, {[0 1]});
class(Pf)
size(Pf)
Pf.Degree
```

Output

```
ans =
    'dpvar'

ans =
     2     3

ans =
     1
```

The structure flag makes the coefficient structure explicit. This is useful when a square variable should use full, not symmetric, coefficients.

MATLAB input

```
yalmip('clear')
Pfull = dpvar(2, 2, {[0 1]}, "full");
class(Pfull)
Pfull.MatrixSize
Pfull.Degree
```

Output

```
ans =
    'dpvar'

ans =
     2     2

ans =
     1
```

Use `Degree=0` for a parameter-independent decision variable stored with grid metadata.

MATLAB input

```
yalmip('clear')
P0 = dpvar(1, [0 1 2], Degree=0);
class(P0)
P0.Degree
P0.ncell()
```

Output

```
ans =
    'dpvar'

ans =
     0

ans =
     2
```

Rate bounds are metadata for later `rhodiff(P)` calls. `dplmi` constrains rate vertices only after an expression such as `rhodiff` has stored rate-vertex rows.

MATLAB input

```
yalmip('clear')
Pr = dpvar(1, {[0 1], [10 20]}, RateBounds=[-1 2; -3 4]);
class(Pr)
Pr.HasRateDependence
Pr.RateBounds
```

Output

```
ans =
    'dpvar'

ans =
  logical
   1

ans =
    -1     2
    -3     4
```

5.3 rhodiff

Syntax

Call forms

```
D = rhodiff(P)
D = rhodiff(P, rb)
```

Arguments and options

`rhodiff(P)` uses `P.RateBounds`; `rhodiff(P, rb)` supplies rate bounds at the call site, such as `rhodiff(P, [-1 1])`. The output is a discontinuous `dpvar` expression with one coefficient row per rate vertex. In one parameter, a degree-m derivative has degree $m-1$, with degree zero retained as a zero derivative. In multiple parameters, partial derivatives are elevated back into the common tensor degree used by the implementation.

For a one-parameter degree-one coefficient pair P_0, P_1 on a cell of width h, the rate-dependent derivative row is

$$\dot{P}(\rho, \dot{\rho}) = \dot{\rho}\,\frac{P_1 - P_0}{h}.$$

The implementation stores one row for each allowed rate vertex.

Example

When rate bounds live on the object, `rhodiff(P)` uses them directly.

MATLAB input

```
yalmip('clear')
P = dpvar(2, {[0 1 2]}, "symmetric", RateBounds=[-1 1]);
D = rhodiff(P);
class(D)
D.Degree
D.IsContinuous
size(D.coeffs(1))
D.RateBounds
```

Output

```
ans =
    'dpvar'

ans =
     0

ans =
  logical
   0

ans =
     2     1

ans =
    -1     1
```

For a two-parameter object, the rate-vertex rows enumerate all lower/upper combinations from the two rows of `RateBounds`.

MATLAB input

```
yalmip('clear')
Q = dpvar(1, {[0 1], [10 20]}, RateBounds=[-1 1; -2 2]);
E = rhodiff(Q);
class(E)
E.Degree
size(E.coeffs([1 1]))
E.RateBounds
```

Output

```
ans =
    'dpvar'

ans =
     1

ans =
     4     4

ans =
    -1     1
    -2     2
```

5.4 `bernsteinTable`

Syntax

Call forms

```
T = bernsteinTable(P)
T = bernsteinTable(P, cellSubs)
T = bernsteinTable(P, "oneLine")
T = bernsteinTable(P, cellSubs, "oneLine")
```

Arguments and options

`bernsteinTable` inspects the local Bernstein coefficient rows of a `dpvar`. `cellSubs` selects one physical cell; omitting it lists all cells. The only text option is `"oneLine"`. Ordinary rows contain the local label, basis, physical-node flag, and symbolic or numeric value. A rate-dependent derivative also includes `RateVertexIndex` and `RateVertex` columns.

Example

MATLAB input

```
yalmip('clear')
P = dpvar(1, {[0 1]});
T = bernsteinTable(P, "oneLine");
disp(T)
```

Output

```
CellSubscript       Expression
-------------    ----------------

    {[1]}        "a*internal(1) + (1-a)*internal(2)"
```

`bernsteinTable` is an inspection method, not a solver call. Function options other than `"oneLine"`, invalid cell selectors, and inconsistent rate-vertex rows are rejected with the following error identifier:

Call forms

```
dpvar:InvalidBernsteinTableInput
```

5.5 Affine And Mixed Algebra

Syntax

Call forms

```
R = P + Q
R = P + A
R = A + P
R = P + M
R = M + P
R = P - Q
R = P - A
R = A - P
R = +P
R = -P
R = A * P
R = P * A
R = M * P
R = P * M
```

Arguments and options

- `dpvar`: affine addition and subtraction with compatible size, grid, and rate metadata.
- Coefficient-backed `dpmat`: mixed known/decision algebra for ordinary `dpvar` expressions, including products where only one side contains decisions.
- Numeric scalar or matrix: promoted as constant coefficient data for ordinary `dpvar` expressions when finite real and size-compatible.
- Affine `sdpvar` matrix: promoted as parameter-independent affine data when real, linear, and size-compatible.
- Rate-vertex derivative rows: supported in the tested derivative and known-data combinations used by `rhodiff` LMI residuals.

- Metadata-only rate-dependent **dpvar** products are unsupported until **rhodiff** has materialized rate-vertex rows.

Example

The affine sum with a numeric matrix keeps the same grid and degree.

MATLAB input

```
yalmip('clear')
P = dpvar(2, {[0 1]}, "symmetric");
S = P + eye(2);
class(S)
size(S)
S.Degree
S.ContainsDecision
```

Output

```
ans =
    'dpvar'

ans =
     2     2

ans =
     1

ans =
  logical
   1
```

Known-data products are allowed when only one side contains decisions. The product degree is the sum of the known-data degree and the decision degree.

MATLAB input

```
yalmip('clear')
P = dpvar(2, {[0 1]}, "symmetric");
A = dpmat({[0 1]}, {eye(2), 2*eye(2)}, Degree=1);
K = A * P;
R = P * A;
class(K)
class(R)
K.Degree
R.Degree
```

Output

```
ans =
    'dpvar'

ans =
    'dpvar'

ans =
     2
```

```
ans =
     2
```

5.5.1 Operator-level entries

`plus`. Forms an affine sum of compatible `dpvar`, numeric, affine `sdpvar`, or coefficient-backed `dpmat` operands.

Syntax

Call forms

```
R = P + Q
R = P + A
R = A + P
R = P + M
R = M + P
```

`minus`. Forms an affine difference and preserves operand order; the same promotion and grid compatibility rules as `plus` apply.

Syntax

Call forms

```
R = P - Q
R = P - A
R = A - P
R = P - M
R = M - P
```

`uplus` and `uminus`. Unary plus preserves the expression; unary minus negates every coefficient expression.

Syntax

Call forms

```
R = +P
R = -P
```

`mtimes`. Supports products with known numeric or coefficient data when decision dependence occurs on at most one side. Products of two decision expressions, or products with rate dependence on both sides, are rejected to preserve affine modeling.

Syntax

Call forms

```
R = A * P
R = P * A
R = M * P
R = P * M
```

Example

MATLAB input

```
yalmip('clear')
P = dpvar(1, {[0 1]});
class(+P)
class(-P)
size(2 * P)
```

Output

```
ans =
    'dpvar'

ans =
    'dpvar'

ans =
     1     1
```

5.6 Matrix Methods

Syntax

Call forms

```
sz = size(P)
n = length(P)
n = height(P)
n = width(P)
n = numel(P)
n = ndims(P)

Q = P.'
Q = P'
Q = vec(P)
Q = diag(P)
Q = diag(P, k)
Q = reshape(P, sz)
Q = reshape(P, m, n)
```

```
Q = tril(P)
Q = triu(P)
Q = trace(P)
Q = sum(P)
Q = sum(P, dim)
Q = sum(P, "all")
Q = mean(P)
Q = mean(P, dim)
Q = mean(P, "all")
Q = cumsum(P)
Q = cumsum(P, dim)
Q = flip(P)
Q = flip(P, dim)
Q = flipud(P)
Q = fliplr(P)
Q = rot90(P)
Q = rot90(P, k)
Q = repmat(P, ...)
Q = [P, R]
Q = [P; R]
Q = cat(dim, P, R, ...)
Q = blkdiag(P, R, ...)
tf = isequal(P, R, ...)
```

Arguments and options

These methods preserve Bernstein grid metadata and act coefficient-wise on the matrix payloads. Dimension arguments follow MATLAB-style calls such as `sum(P,2)`, `reshape(P,[3 2])`, and `diag(P,1)`. Concatenation and `blkdiag` may combine `dpvar`, coefficient-backed `dpmat`, affine `sdpvar`, and numeric blocks when the matrix dimensions, grid bounds, degree elevation, and rate metadata are compatible.

Example

Shape-changing operations return new `dpvar` objects and preserve the decision-backed coefficient graph.

MATLAB input

```
yalmip('clear')
P = dpvar(2, 3, {[0 1]}, "full");
Tt = P.';
class(Tt)
size(Tt)
V = vec(P);
class(V)
size(V)
R = reshape(P, [3 2]);
class(R)
size(R)
```

Output

```
ans =
    'dpvar'

ans =
     3     2

ans =
    'dpvar'

ans =
     6     1

ans =
    'dpvar'

ans =
     3     2
```

Summary and assembly calls use the same coefficient-wise mechanism.

MATLAB input

```
yalmip('clear')
P = dpvar(2, 3, {[0 1]}, "full");
S = sum(P, 2);
B = blkdiag(P(1:2,1:2), eye(2));
class(S)
class(B)
size(S)
size(B)
```

Output

```
ans =
    'dpvar'

ans =
    'dpvar'

ans =
     2     1

ans =
     4     4
```

5.6.1 Additional matrix and protocol methods

`transpose` and `ctranspose`. `transpose` applies `P.'`; `ctranspose` applies `P'`. They preserve the YALMIP coefficient graph and transpose every local payload.

Syntax

Call forms

```
Q = transpose(P)
Q = ctranspose(P)
Q = P.'
Q = P'
```

squeeze. Applies the package's two-dimensional squeeze convention to every local decision payload.

Syntax

Call forms

```
Q = squeeze(P)
```

horzcat and vertcat. The bracket forms concatenate compatible `dpvar`, known-data, affine `sdpvar`, and numeric blocks while preserving compatible grid, degree, and rate metadata.

Syntax

Call forms

```
Q = horzcat(P, R)
Q = vertcat(P, R)
Q = [P, R]
Q = [P; R]
```

subsref and subsasgn. `subsref` handles matrix indexing and dot access; `subsasgn` assigns numeric, `dpvar`, coefficient-backed `dpmat`, or affine `sdpvar` blocks.

Syntax

Call forms

```
Q = subsref(P, S)
P = subsasgn(P, S, X)
Q = P(rows, cols)
P(rows, cols) = X
```

end. Supplies the last row or column index for expressions such as `P(1:end,end)`. The related `numArgumentsFromSubscript` method preserves scalar subscript-result behavior for MATLAB dot access.

Syntax

Call forms

```
idx = end(P, k, n)
n = numArgumentsFromSubscript(P, S, ctx)
Q = P(1:end, end)
```

Example

MATLAB input

```
yalmip('clear')
P = dpvar(2, {[0 1]}, "full");
size([P, P])
size(P(1:end, end))
size(squeeze(P))
```

Output

```
ans =
     2     2

ans =
     2     1

ans =
     2     2
```

5.7 Indexing And Assignment

Syntax

Call forms

```
Q = P(rows, cols)
value = P.PropertyName
P(rows, cols) = numericValue
P(rows, cols) = Q
P(rows, cols) = A
P(rows, cols) = X
```

Arguments and options

Matrix indexing slices the matrix payload across every physical cell and local coefficient. Assignment accepts numeric blocks, compatible `dpvar` blocks, compatible coefficient-backed `dpmat` blocks, and affine `sdpvar` blocks. Assignment may elevate degrees and propagate compatible rate metadata. Use ordinary MATLAB subscripts such as `P(1:2,1)` or `P(:,2)=A`.

Example

Indexing returns a smaller `dpvar`; assignment updates the selected matrix entries across every local coefficient.

MATLAB input

```
yalmip('clear')
P = dpvar(2, 3, {[0 1]}, "full");
col = P(:, 2);
P(:, 1) = 0;
class(col)
size(col)
size(P)
```

Output

```
ans =
    'dpvar'

ans =
     2     1

ans =
     2     3
```

5.8 Comparisons To `dplmi`

Syntax

Call forms

```
C = P <= 0
C = P >= 0
C = lhs <= rhs
C = lhs >= rhs
```

Arguments and options

The comparison overloads construct `dplmi` objects. The expression must be square and coefficient-wise symmetric or Hermitian after assembly. `rhs` is usually `0`, but compatible numeric, `dpmat`, affine `sdpvar`, and `dpvar` expressions may appear in the residual. `dpvar` comparisons do not solve an optimization problem by themselves.

Example

Comparison creates a `dplmi` wrapper with one stored constraint for each physical cell and local Bernstein coefficient.

MATLAB input

```
yalmip('clear')
P = dpvar(2, {[0 0.5 1]}, "symmetric");
Cneg = P <= 0;
class(Cneg)
numel(Cneg.Constraints)
Cpos = P >= 0;
class(Cpos)
numel(Cpos.Constraints)
```

Output

```
ans =
    'dplmi'

ans =
     4

ans =
    'dplmi'

ans =
     4
```

5.9 Limits

- Constructor-created `dpvar` objects currently support `Degree=0` and `Degree=1`.
- Products may contain decision variables on at most one side; nonlinear decision products such as `P * Q` are outside this slice.
- Function-only `dpmat` operands cannot enter `dpvar` algebra unless they were constructed with explicit Bernstein evidence.
- `rhodiff` of an existing rate-vertex derivative expression is not part of the current public workflow.

5.9.1 See Also

`dpvar`, `rhodiff`, `bernsteinTable`, `dplmi`, `dpmat`.

6

dplmi Reference

`dplmi` stores direct coefficient-wise YALMIP constraints assembled from square symmetric `dpvar` expressions. It is the package boundary between Bernstein coefficient objects and ordinary YALMIP solver calls.

6.1 Lookup

Task	Entry
Construct wrapper	`dplmi`, `<=/>=`
Count constraints	direct coefficient constraints
Export to YALMIP	`toYalmip`
Solve	solver-facing use, Examples

6.2 Construction And Comparisons

Syntax

Call forms

```
C = dplmi(expr, relation)
C = dplmi(expr, relation, relaxLemma=false, UsePolya=false, PolyaDegree=0)
C = lhs <= rhs
C = lhs >= rhs
```

Arguments and options

`expr` must be a square `dpvar` expression whose coefficient matrices are symmetric or Hermitian. The `relation` input is `"<="` or `">="`. Comparisons such as `lhs >= rhs` are the ordinary user-facing entry point and support compatible numeric, `dpmat`, affine `sdpvar`, and `dpvar` right-hand sides. Direct `dplmi(expr, relation)` construction is useful for advanced assembly or tests. Reserved options must stay at their defaults: `relaxLemma=false`, `UsePolya=false`, and `PolyaDegree=0`; nondefault values currently raise reserved-option errors.

Example

The direct constructor is equivalent to the comparison path when the relation and reserved options are explicit.

MATLAB input

```
yalmip('clear')
P = dpvar(2, {[0 1]}, "symmetric");
C = dplmi(P, ">=", relaxLemma=false, UsePolya=false, PolyaDegree=0);
class(C)
numel(C.Constraints)
C.RelaxLemma
C.UsePolya
C.PolyaDegree
```

Output

```
ans =
    'dplmi'

ans =
     2

ans =
  logical
   0

ans =
  logical
   0

ans =
     0
```

6.3 Direct Coefficient Assembly

For each physical cell, local Bernstein coefficient, and rate-vertex row if present, `dplmi` creates one YALMIP semidefinite constraint. For an ordinary expression with N_c physical cells and N_b Bernstein coefficients per cell, the number of stored constraints is $N_c N_b$. For a `rhodiff` expression with N_v rate vertices, the count becomes $N_c N_b N_v$.

6.4 `toYalmip`

Syntax

Call forms

```
F = toYalmip(C)
F = C.toYalmip()
```

Arguments and options

`toYalmip` concatenates the stored constraint cells into a YALMIP constraint array. It has no package-specific options. The output is suitable for `optimize`, concatenation with other YALMIP constraints, or ordinary YALMIP inspection.

Example

The function-call and dot-call forms return equivalent YALMIP constraint arrays.

MATLAB input

```
yalmip('clear')
P = dpvar(2, {[0 1]}, "symmetric");
C = P >= 0;
class(C)
numel(C.Constraints)
F1 = toYalmip(C);
F2 = C.toYalmip();
isa(F1, "lmi") || isa(F1, "constraint")
length(F1)
isa(F2, "lmi") || isa(F2, "constraint")
length(F2)
```

Output

```
ans =
    'dplmi'

ans =
     2

ans =
  logical
   1

ans =
     2

ans =
  logical
   1

ans =
     2
```

After this boundary, the package is no longer assembling Bernstein data; YALMIP owns solver choice, diagnostics, and optimization status.

6.5 See Also

`dpvar`, `rhodiff`, `toYalmip`, `optimize`.

6.6 Solver-Facing Use

`dplmi` does not own solver settings or wrap `optimize`. After `toYalmip`, use ordinary YALMIP:

Call forms

```
opts = sdpsettings("solver", solverName, "verbose", 0);
sol = optimize([C1.toYalmip, C2.toYalmip], objective, opts);
```

The solver selected by YALMIP must be available on the MATLAB path. The current test suite prefers MOSEK when available and otherwise falls back to LMILAB for smoke coverage.

7

Examples

7.1 Rate-Dependent Block LMI Solver Smoke

This example is adapted from `+tests/+dplmi/test_solver_smoke.m`. The goal is to find a parameter-dependent $P(\rho)$ and scalar γ such that, on every Bernstein coefficient and rate vertex,

$$\begin{bmatrix} \dot{P} + PA + A^\top P & PB & C^\top \\ B^\top P & -\gamma I & D^\top \\ C & D & -\gamma I \end{bmatrix} \preceq 0, \qquad P(\rho) \succeq 0.$$

The derivative term $\dot{P}$ is built with `rhodiff(P, [-1 1])`. The comparison operators create `dplmi` wrappers, and `toYalmip` hands the coefficient-wise constraints to YALMIP.

MATLAB input

```
yalmip('clear')
grid = linspace(0, 1, 2);
A = dpmat(grid, @(x) [-1, 0.5; -1, -2] + ...
    x * [-1.3, -20; 2, -10], Degree=1);
B = dpmat(grid, @(x) [1, -4; -1, -1] + ...
    x * [2.2, 0.5; -6, -5], Degree=1);
Cmat = eye(2);
Dmat = zeros(2);

P = dpvar(2, grid);
diffP = rhodiff(P, [-1 1]);
gamma = dpvar(1, grid, Degree=0);
E1 = [diffP + P * A + A' * P, P * B, Cmat';
      B' * P, -gamma * eye(2), Dmat';
      Cmat, Dmat, -gamma * eye(2)] <= 0;
E2 = P >= 0;
```

MATLAB input (continued)

```
objective = gamma.LocalValues{1}{1};
solver = 'lmilab';
if exist('mosekopt', 'file') ~= 0
    solver = 'mosek';
end
opts = sdpsettings('solver', solver, 'verbose', 0);
sol = optimize([E1.toYalmip, E2.toYalmip], objective, opts);

numel(E1.Constraints)
numel(E2.Constraints)
sol.problem
isfinite(value(objective))
```

Output

```
ans =
     6

ans =
     2

ans =
     0

ans =
  logical
   1
```

8

Current Limits

- `dplmi` currently assembles direct coefficient-wise constraints. Relaxation-lemma and Polya workflows are reserved future layers, not supported user workflows in this manual.
- Package-owned solver wrappers, strictness-margin diagnostics, residual evidence, and advanced relaxation workflows are not part of the current public API.
- `dpvar` is an affine YALMIP/Bernstein expression object. It does not support nonlinear decision products.
- `dpmat` function-only objects evaluate exactly, but coefficient algebra requires Bernstein coefficient evidence.
- `dpbase` is documented for architecture and advanced inspection. Primary modeling workflows should use `dpmat`, `dpvar`, and `dplmi`.

References

[1] R. T. Farouki, “The Bernstein polynomial basis: A centennial retrospective,” *Computer Aided Geometric Design*, vol. 29, no. 6, pp. 379–419, 2012. doi:10.1016/j.cagd.2012.03.001.

[2] M. Zettler and J. Garloff, “Robustness analysis of polynomials with polynomial parameter dependency using Bernstein expansion,” *IEEE Transactions on Automatic Control*, vol. 43, no. 3, pp. 425–431, 1998. doi:10.1109/9.661615.

[3] I. Masubuchi, A. Kume, and E. Shimemura, “Spline-type solution to parameter-dependent LMIs,” in *Proceedings of the 37th IEEE Conference on Decision and Control*, vol. 2, 1998. doi:10.1109/CDC.1998.758549.

[4] Y. Xu and F. Jabbari, “Spline-based parameter varying output feedback synthesis with improved L_2 gain,” in *2024 IEEE 63rd Conference on Decision and Control*, pp. 1455–1460, 2024. doi:10.1109/CDC56724.2024.10886461.

[5] G. Chesi, “LMI techniques for optimization over polynomials in control: a survey,” *IEEE Transactions on Automatic Control*, vol. 55, no. 11, pp. 2500–2510, 2010. doi:10.1109/TAC.2010.2046926.